

## Architecture Decisions

---

Version: 1.2.0 | Updated: 2026-06-24

Key technical choices made in this project, with rationale. This document is intended for anyone reviewing the codebase who wants to understand not just what was built, but why.

---

### 1. Local-First Design (Ollama, No External API)

---

**Decision:** All LLM inference and embedding runs locally via Ollama. No OpenAI, Anthropic, or other cloud API dependency in the core stack.

**Why:** - Forces genuine understanding of model behavior at the infrastructure level — latency, memory pressure, model selection tradeoffs - Eliminates API cost as a variable during experimentation - Relevant to financial services contexts where data residency and network egress are real constraints, not afterthoughts - Makes the system runnable in air-gapped or restricted environments

**Tradeoff:** Inference quality is lower than frontier models. This is a deliberate constraint, not a limitation. The goal is substrate knowledge, not benchmark performance.

**Would change for production:** Add LiteLLM as a provider abstraction layer. Local inference for development and sensitive data; cloud inference (with appropriate data handling) for production quality. The architecture is designed to make this substitution straightforward — Ollama client is isolated in `ollama_client.py`.

---

### 2. Qdrant as Vector Store (replaced ChromaDB)

---

**Decision:** Qdrant 1.17.0, running as a separate process on port 6333. ChromaDB was the original choice and was replaced in production.

**Why the switch:** ChromaDB crashed at 32,285 chunks during the SEC 10-K corpus ingest. Not a configuration issue — a hard failure at scale. Qdrant was stable at 100,659 chunks with zero failures across multiple full re-ingests.

**Why Qdrant specifically:** - Written in Rust — near-zero GC overhead vs ChromaDB's Python-based memory model - Native metadata filtering via `Filter / FieldCondition` — firm and year filters run at the vector search layer, not post-hoc on results - Production upgrade path: sharding, replication, quantization, gRPC (port 6334) - Separate process model — independently restartable, independently scalable, maps directly to a Docker container in cloud deployment

**The four-process pattern is deliberate:** Browser → FastAPI → Qdrant → Ollama. Each process independently restartable. The separation that looks like operational overhead in a personal lab is the same separation that enables horizontal scaling in production — add Qdrant nodes, add uvicorn workers, point at shared Qdrant cluster. No architecture change required.

**Tradeoff:** Qdrant requires a separate process and persistent storage directory ( `~/qdrant_storage/` ). ChromaDB ran in-process with zero operational overhead. The stability gain at scale makes this tradeoff non-negotiable.

**Would change for production:** Same Qdrant, configured for replication and S3-backed snapshots. Alternatively Qdrant Cloud for managed hosting. The `qdrant_store.py` adapter makes the substitution a configuration change.

---

### 3. Embedding Model: `nomic-embed-text`

---

**Decision:** `nomic-embed-text` as the default embedding model (768 dimensions, cosine similarity).

**Why:** - Best CPU/unified memory performance among locally available models at time of selection — low latency without GPU - Produces meaningful semantic representations for the document types in scope - Passes embedding arithmetic tests (King – Man + Woman ≈ Queen), a practical signal of semantic quality

**Considered:** `bge-large-en` — approximately 5% quality improvement on retrieval benchmarks, but roughly 2x slower. The tradeoff didn't justify it.

**Would change for production:** Benchmark against the specific corpus and query patterns in scope. Embedding model selection is highly workload-dependent.

---

### 4. Citation Logic Embedded in API, Not Modularized

---

**Decision:** Citation extraction and formatting lives in `api.py` rather than as a separate module.

**Why:** - Citation logic is tightly coupled to the response pipeline — it operates on LLM output immediately before it's returned - Separating it adds indirection without architectural benefit at this scale - Simpler deployment: one file to update, no import chain to maintain

**Tradeoff:** `api.py` is longer as a result.

**Would change for production:** With multiple citation formats or output channels, a dedicated citation module with a clean interface would be worth the overhead.

---

## 5. Semantic Chunking Over Fixed-Size Chunking

---

**Decision:** Boundary-aware semantic chunking. The previous implementation used fixed character counts.

**Why:** - Fixed chunking fragments natural language units — a sentence split across two chunks degrades retrieval significantly - Document structure carries semantic information: section headers, paragraph breaks, sentence boundaries are meaningful signals - Measured improvement: ~25–40% accuracy improvement on document queries

**Implementation:** Three-tier fallback — semantic boundary detection → sentence splitting → character fallback.

**Tradeoff:** Variable-length chunks create some unpredictability in context window usage. Handled via explicit context window management in the retriever.

---

## 6. Metadata Filtering at the Vector Layer

---

**Decision:** Firm and year filters are applied as Qdrant `FieldCondition` filters at query time, not as post-hoc filtering on results.

**Why:** - Post-hoc filtering wastes retrieval budget — you retrieve top-K then discard most of them. Vector-layer filtering retrieves top-K from the filtered subset. - Qdrant's native Filter API has zero measurable latency overhead. - Metadata (firm, year) is parsed from filenames at ingest time ( `_parse_firm_year()` in `pipeline.py` ) and stored as Qdrant payload fields. No manual tagging required.

**Pattern:** `Goldman_Sachs_10K_2026-02-25.htm` → `firm=Goldman Sachs, year=2026` parsed automatically. Filter adds zero latency. Filter is optional — omitting it runs cross-corpus retrieval.

---

## 7. Page-Aware PDF Chunking via pdfplumber (*implemented Beta*)

---

**Decision:** pdfplumber as primary PDF extractor, inserting `[PAGE_N]` markers at each page boundary. pypdf retained as fallback.

**Why pdfplumber over pypdf:** - pypdf extracts flat text with no page boundary information - pdfplumber provides page-by-page extraction with reliable boundary detection - `[PAGE_N]` markers flow through the chunking pipeline into Qdrant payload ( `page` field) and `chunk_id` ( `filename::page-N::chunk-M` )

**Pipeline:** `loaders._extract_pdf()` → `[PAGE_N]` markers in text → `pipeline.py` extracts page numbers → stored in Qdrant payload → surfaced in `RetrievedDoc.page` → rendered in UI citation footnotes with `Open ↗` link.

**Null pages are expected** for: non-PDF files (PPTX, DOCX), scanned PDFs with no text layer, and continuation chunks that start mid-page.

**PDF viewer — click citation → scroll to page** is tracked as Source Dive (pre-Jessica gate item). Page numbers are ready in the backend; the viewer frontend is the remaining work.

---

## 8. Conversation Memory: Sliding Window of 10 Turns

---

**Decision:** Last 10 conversation turns maintained as context for follow-up questions.

**Why:** - Covers most realistic conversational patterns without excessive context window consumption - Stateless between sessions by design — no persistence layer required

**Tradeoff:** Long research sessions lose early context. Vector-based memory store (storing conversation summaries as embeddings) would be more appropriate for persistent multi-session memory.

---

## 9. JSONL Baseline for Deterministic Testing

---

**Decision:** A JSONL retrieval path exists alongside the Qdrant path, returning deterministic results without embedding inference.

**Why:** - Embedding models are nondeterministic under some conditions and slow in CI - Allows testing the full API response pipeline without Ollama or Qdrant running - Unit tests use the JSONL path; integration tests use Qdrant

---

## 10. Single HTML File Frontend

---

**Decision:** The entire UI is `front_end/rag_studio.html` — one file, ~1,900 lines, no build step.

**Why:** - Zero build toolchain. Clone and open. Works immediately. - No npm, no webpack, no node\_modules. Nothing to break on a new machine. - Consistent with the "runnable in under 30 minutes" QUICKSTART promise.

**Current UI features (v2.0.1):** Corpus management (create, rename, delete, stats, inspect), file upload with ingest progress, chat with inline citations and inline code rendering (backtick and fenced block), corpus/model change separators, auto-linkify of URLs and file paths, clickable citation filenames, corpus selector with alphabetical sort, Retrieval Mix slider (hybrid\_alpha, Literal↔Conceptual), query settings sidebar, system prompt hot-reload via `/prompt/reload`.

**Tradeoff:** A single ~2,200-line file is harder to navigate than a componentized React app. The tradeoff is deliberate — operational simplicity over developer ergonomics at this scale.

**Would change for production:** React with component library for multi-user Beta. The `ui_architecture.md` doc describes the target left-bar layout.

---

## 15. Manifest-Driven Command Installation

---

**Decision:** A single manifest is the source of truth for all command routing, deployment, and installation. The installer reads the manifest to register commands — no hardcoded paths anywhere else.

**Why:** The previous approach used one-shot install scripts that couldn't be re-run when new commands were added, so adding a command meant editing shell configuration by hand — fragile, error-prone, non-reproducible.

The manifest-driven approach means: - A single new command can be installed by looking up its path in the manifest - The active command set can be verified against the manifest at any time - A fresh clone on a new machine installs every command from the same manifest, guaranteeing consistency - The manifest makes the command registry explicit and auditable

**Tradeoff:** Adding a new command now requires a manifest update before deployment. This is by design — the manifest is the deployment contract, not an afterthought.

## Production Hardening (cross-cutting)

---

What the system would need before any network exposure:

- **Authentication:** The API has no auth. Fine for localhost; needs API key validation before any network exposure.
- **Observability:** LLM latency, retrieval score distributions, and refusal rates are the three most important things to instrument first.
- **Async ingestion:** Currently synchronous and blocking. Needs a queue-backed async pipeline at meaningful document volumes.
- **Vector store backup:** Qdrant storage ( `~/qdrant_storage/` ) is a derived artifact but expensive to regenerate. Treat it like a database — back it up. In production: S3-backed Qdrant snapshots on a schedule.

---

## 11. CrossEncoder Reranker: Two-Stage Retrieval

---

**Decision:** CrossEncoder `ms-marco-MiniLM-L-6-v2` as a reranking pass after vector search, before prompt assembly.

**Why:** The bi-encoder (nomic-embed-text) compresses query and chunk independently into fixed vectors. Compression loses nuance — "AI governance committee" and "Firmwide Artificial Intelligence Risk and Controls Committee" are semantically equivalent but vector-distant. This is vocabulary mismatch.

The CrossEncoder reads query + chunk concatenated as a single input, with full attention across both. It scores relevance directly rather than approximating via cosine distance. Vocabulary mismatch largely disappears.

**Two-stage architecture:** - Stage 1: Qdrant HNSW vector search — fast, retrieves top-K candidates - Stage 2: CrossEncoder reranker — slower, reorders by true relevance

**Model selection:** `ms-marco-MiniLM-L-6-v2` - 22M parameters, 90MB — loads in <1s on Apple Silicon - Fine-tuned on MS MARCO (500K passage relevance pairs) — directly applicable to "rank these passages for this query" - Best latency/quality tradeoff in the MiniLM family - Knowledge-distilled from BERT-large — 6 transformer layers, preserves most quality at fraction of the depth

**Implementation:** `rag_core.py` — lazy load with graceful fallback if `sentence-transformers` unavailable. Zero impact on existing behavior if reranker fails to load.

**Tradeoff:** ~1-2s additional latency per query for CrossEncoder inference on top-K candidates. Acceptable at current corpus sizes.

---

## 12. Atomic `--force` Ingest

---

**Decision:** `--force` flag in the ingest pipeline forces reprocessing of all files regardless of whether they appear unchanged in the Qdrant manifest.

**Why:** Without `--force`, files already in Qdrant are skipped (MD5-based skip logic). When chunk format changes or a corpus needs a clean rebuild, `--force` bypasses the skip check and re-embeds everything.

**Clarification on atomic wipe:** `--force` does NOT wipe the Qdrant collection itself — it only bypasses the per-file skip check. A full corpus wipe (delete collection + re-ingest from scratch) is triggered separately via `DELETE /corpus/{name}?confirm=yes` in `api.py`, or by the `ais_start` help corpus auto-ingest which uses `--force` after collection existence check.

**Implementation:** `pipeline.py ingest_corpus()` — when `force=True`, `abs_path` in `qdrant_source_paths` check is bypassed. Files are re-chunked, re-embedded, and upserted regardless of prior state. Qdrant's upsert is idempotent — duplicate chunk IDs are overwritten cleanly.

**Also used by:** `scripts/start.sh` help corpus background refresh on every start.

---

## 13. Session Continuity via Structured State Packets

---

**Decision:** Operator workflow uses a PACKET/BUNDLE protocol to maintain full project context across AI-assisted development sessions. Each session ends with a structured markdown PACKET (produced by an internal session-state tool) capturing repo state, active pipeline, decisions made, and session summary. The PACKET is bundled with supporting documents into a zip archive attached to the next session.

**Why:** LLM context windows are finite and session-scoped. Without an explicit continuity mechanism, each session starts cold — the assistant has no memory of prior decisions, naming conventions, or work in progress. The PACKET protocol treats session state as a first-class artifact: structured, versioned, and portable.

The result is an effectively unbounded working memory across sessions. Complex multi-week development work — with branching decisions, evolving standards, and accumulated conventions — can be resumed with full fidelity. The protocol is also self-referential: the PACKET contains the rules for generating the next PACKET.

**Pattern:** a structured PACKET.md is generated, reviewed in session, then bundled with supporting docs and attached to the next thread — full context restored in under two minutes.

**Broader relevance:** This pattern is a prototype for the urCrew personal operating system concept — where structured state packets manage continuity across multiple parallel life and work threads, each with its own agent context. The PACKET/BUNDLE workflow is Phase 0 of that system, built and validated here.

**Tradeoff:** Requires discipline at session end. The session-end protocol codifies this as a non-negotiable checklist.

---

## 14. REST API Design and Publication

---

**Decision:** AIStudio exposes a REST API via FastAPI on port 8000. All UI interactions go through this API — the frontend is a pure client with no direct access to the filesystem, Qdrant, or Ollama.

**Current endpoints (Beta):** - `POST /ask` — RAG query with corpus, model, top-K, temperature, hybrid\_alpha, keywords, conversation history - `POST /corpus/{name}/ingest` — trigger ingest of uploaded files - `GET /corpus/{name}/ingest-status` — streaming ingest progress (files, chunks, elapsed) - `GET /corpora` — list available corpora with metadata - `POST /corpus/create` — create new corpus directory structure - `POST /corpus/{name}/rename` — rename corpus directory + Qdrant collection + trigger re-ingest - `DELETE /corpus/{name}` — move corpus to macOS Trash (recoverable) - `DELETE /corpus/{name}/file/{filename}` — move file to corpus trash - `GET /corpus/{name}/info` — file list, chunk count, size - `POST /corpus/{name}/upload` — upload files to corpus - `POST /prewarm` — warm Ollama model into memory before first query - `GET /source` — serve source documents for citation Open ↗ links - `POST /prompt/reload` — hot-swap system prompt without restart (clears memoized `_SYSTEM_PROMPT_BASE`) - `GET /health` — liveness check (used by start.sh health-check poll) - `GET /models` — list available Ollama models

**Why a clean API separation matters:** The single-page frontend communicates exclusively via `fetch()` calls to this API. This means AIStudio's RAG capabilities are immediately consumable by any HTTP client — scripts, notebooks, other applications, or future agent orchestration layers. No SDK required.

**Planned:** Full `API_DOC.md` with request/response schemas and error codes published alongside the API. Swagger/OpenAPI auto-docs via FastAPI's built-in support ( `/docs` endpoint). This is a v2.0 item — the API surface is stable at Beta but not yet formally documented.

---

## 16. Hybrid Retrieval: BM25 + Vector Score Combination (M2.A)

---

**Decision:** Hybrid retrieval combining Qdrant vector search with BM25 sparse retrieval, score-combined via a weighted `hybrid_alpha` parameter. Exposed as a per-query slider ("Retrieval Mix") in the UI and as a `--alpha` flag in the benchmark harness.

**Why:** Pure vector retrieval (semantic similarity) retrieves conceptually related chunks but can miss exact-match keyword queries — e.g. a query for "Goldman Sachs CET1 ratio 2024" may return semantically related risk content instead of the specific number. BM25 is token-based and excels at exact-match recall. Combining both captures the strengths of each.

**Empirical finding:**  $\alpha=0.5$  (equal blend) +  $K=10$  achieves 14/14 on the demo corpus benchmark. Pure vector ( $\alpha=0.0$ ,  $K=5$ ) achieves 13/14 — the two failures were both list-format answers where the model dropped citations. At  $\alpha=0.5$ , retrieval precision is higher and citation compliance improves.

**Implementation:** `scoring.py` — pure functions `normalize_minmax()` and `combine_hybrid()`. `qdrant_store.py` v1.1.0 adds `query_bm25()` and `_ensure_text_index()`. `rag_core.py` v1.5.0 accepts `hybrid_alpha` kwarg. `api.py` v1.6.0 exposes `hybrid_alpha` on `AskRequest`. UI slider: Literal=left ( $\alpha=1.0$  BM25), Conceptual=right ( $\alpha=0.0$  vector) — display shows actual  $\alpha$  sent.

**UI convention note:** The slider direction inverts the raw parameter — `hybrid_alpha: 1 - parseFloat(alphaInput.value)`. Documented to prevent future confusion.

**Tradeoff:** BM25 index must be built on Qdrant collection creation. `_ensure_text_index()` is idempotent — safe to call on existing collections. Minimal overhead.

**Default:** Backend default is pure vector ( $\alpha=None$ ). UI default is  $\alpha=0.5$ . The separation allows the API to remain backward-compatible while the UI surfaces the recommended setting.

---

## 17. Document-Head Extraction Normalizer

---

**Decision:** At ingest time, `.htm/.html/.xhtml` files (SEC XBRL, ESEF iXBRL) have a `[Document: <entity> FY<year>]` prefix injected into every chunk before embedding.

**Why:** Financial filings use anaphora pervasively — "the Company", "the Firm", "the registrant" rather than the entity name. A chunk reading "the Company's CET1 ratio was 15.7%" is semantically indistinguishable from the same sentence in any other filing at embedding time. The prefix resolves anaphora at the embedding layer: "Goldman Sachs FY2024 — the

Company's CET1 ratio was 15.7%" embeds near other Goldman Sachs content, not near generic CET1 content.

**Implementation:** `pipeline.py` `_extract_document_metadata()` — extracts

`dei:EntityRegistrantName` and `dei:DocumentFiscalYearFocus` from inline XBRL tags.

Fallback chain: authoritative tag → scan → filename stem. Non-markup files (PDF, DOCX, MD) receive null returns — no prefix, no overhead. Controlled by `AISTUDIO_DOC_HEAD_NORMALIZER` env var.

**Measured impact:** Entity extraction 9/9, year extraction 9/9 on ESEF corpus. SEC 10-K corpus: enables temporal trend queries at K=5 that previously required K=20+ to retrieve both the entity and the relevant year.

**Tradeoff:** Prefix adds ~8–12 tokens per chunk. Context window budget is slightly reduced. Negligible at current chunk sizes (1,200 chars, ~300 tokens).

---

## 18. System Prompt Architecture: File-Based with Hot-Reload

---

**Decision:** The system prompt lives in `prompts/system.txt`, loaded at runtime by `api.py` and memoized. A `POST /prompt/reload` endpoint clears the cache, enabling prompt updates without restarting the backend.

**Why:** Embedding the system prompt in code (`api.py`) ties prompt iteration to deployment cycles. Separating it into a file makes prompt engineering a first-class workflow: edit the file, hit reload, query immediately. No `ais_start` required.

**Memoization pattern:** `_SYSTEM_PROMPT_BASE` is a module-level variable initialized to `None`. On first query, `api.py` reads and caches the file. `/prompt/reload` sets it back to `None` — next query re-reads from disk.

**Prompt design:** Citation rules use positive + negative examples, not just ban lists. Small models (llama3.1:8b) respond better to CORRECT/WRONG pairs than to prohibition lists alone — prohibitions suppress the cited behavior but can inadvertently suppress the citation marker entirely. The positive example pattern ("Net revenue increased 12% in fiscal 2024 [2]") anchors the correct output format explicitly.

**Tradeoff:** System prompt is not version-controlled in git (it's a runtime file, not source code). Tracked via the BUNDLE/PACKET protocol.